Your AI agents put customer trust at risk

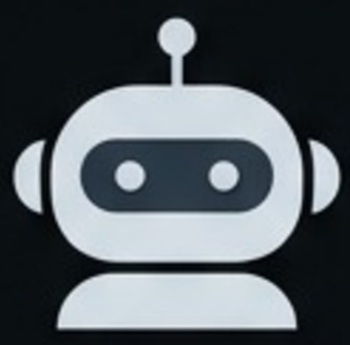
Swipe →

What if an AI agent could modify someone else's data?



User A

Cancel Order #4821



AI Agent

APPLICATION

Does Order #4821 exist?

YES

ORDER #4821

Owner: User B

Status: CANCELLED

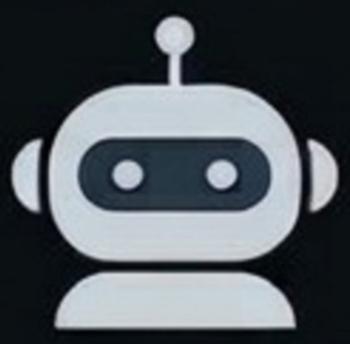
This is BOLA.
Broken Object Level Authorization

The application checks whether the order exists,
but not whether User A is allowed to cancel it.



User A

Cancel Order #4821



AI Agent

APPLICATION

Does Order #4821 exist?

YES

Authorization check missing
(BOLA violation)

ORDER #4821

Owner: User B

Status: CANCELLED

The fix: check authorization before executing the action.

The application must check not just for existence, but for user permission.

1. User A



Cancel Order #4821

2. AI Agent

3. **APPLICATION**

Does Order #4821 exist?

YES

4. Is **User A** allowed to cancel this order? **NO** Request denied

5. **ORDER #4821**

Owner: User B

Status: NOT CANCELLED

Inside the lab

A controlled environment to reproduce and investigate an object-level authorization flaw.

ORDER ID	OWNER ID	STATUS
#1	8 You	pending
#2	2147483646	pending
#3	2147483645	pending

The vulnerable implementation

The following code doesn't verify whether the authenticated actor is authorized to cancel that particular order.

```
def execute(
    self,
    *,
    order_id: int,
    lab_owner_id: int,
) -> Order:
    if not self._order_repository.belongs_to_lab(
        order_id,
        lab_owner_id,
    ):
        raise EntityNotFound(
            f"Order {order_id} was not found."
        )

    order = self._order_repository.get_by_id(order_id)

    if order is None:
        raise EntityNotFound(
            f"Order {order_id} was not found."
        )

    order.cancel()
    self._order_repository.save(order)

    return order
```

Exploiting the vulnerability

Observe what happens when a user attempts to cancel an order belonging to someone else.

ORDER ID	OWNER ID	STATUS
#1	8 You	pending
#2	2147483646	cancelled
#3	2147483645	pending

Reset sets statuses to pending. Orders and IDs are preserved.

Refresh orders

Try it

Select a mode, then ask the agent to cancel an order from the table above.

Vulnerable

Order ownership is not enforced during cancellation.

Hardened

The backend checks whether you can cancel the order.

Orders Agent VULNERABLE

I'm the Orders Agent. Ask me to cancel an order from the list below, then compare both implementations.

cancel order 2

The order with ID 2 has been cancelled. If you need any further assistance, just let me know!

HTTP 200 · Order #2: cancelled · Owner ID: 2147483646

e.g. Cancel order 1844.

Enter to send · Shift+Enter for a new line

The fix in action

After hardening, the unauthorized cancellation request should be rejected.

ORDER ID	OWNER ID	STATUS
#1	8 You	pending
#2	2147483646	pending
#3	2147483645	pending

Reset sets statuses to pending. Orders and IDs are preserved.

Refresh orders

Try it

Select a mode, then ask the agent to cancel an order from the table above.

Vulnerable

Order ownership is not enforced during cancellation.

Hardened

The backend checks whether you can cancel the order.

Orders Agent HARDENED

Hardened mode is active. Try cancelling another user's order and compare the authorization result.

cancel order 2

You are not authorized to cancel this order.

HTTP 403

e.g. Cancel order 1844.

Enter to send · Shift+Enter for a new line

Authorization check

Verify that the application checks ownership or permission before executing the operation.

```python
def execute(
    self,
    actor: Actor,
    order_id: int,
) -> Order:
    order = self._order_repository.get_by_id(order_id)

    if order is None:
        raise EntityNotFound(
            f"Order {order_id} was not found."
        )

    if not self._authorization_policy.is_allowed(
        actor=actor,
        permission=Permission.CANCEL_ORDER,
        resource_owner_id=order.customer_id,
    ):
        raise AuthorizationDenied(
            f"Actor {actor.user_id} is not authorized to cancel "
            f"order {order.id}."
        )

    order.cancel()
    self._order_repository.save(order)

    return order
```

Explore more

Inside Agentic Security

cybersecuritypainintelligence.com